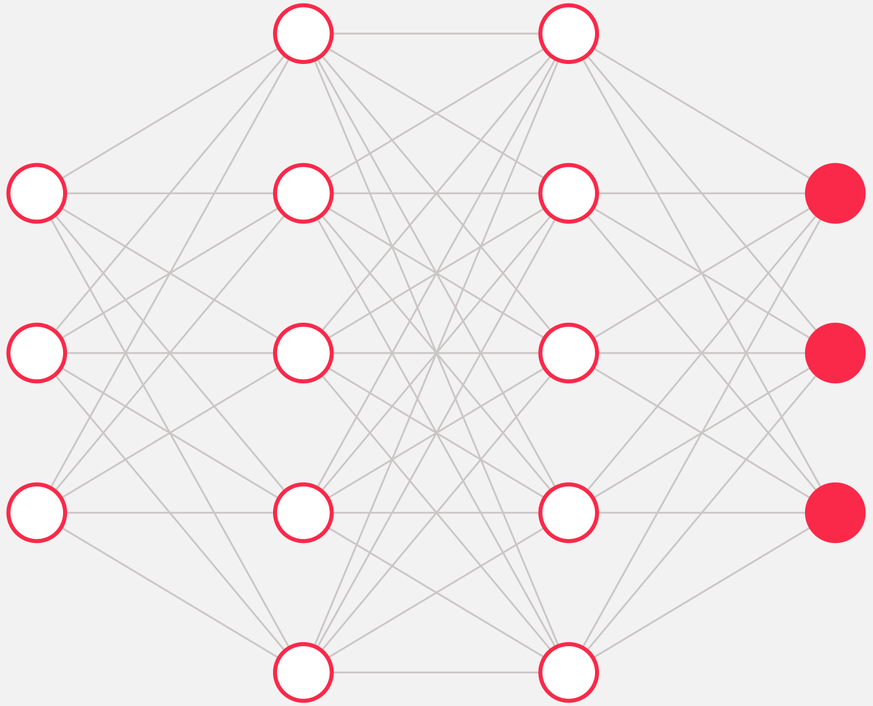
IA Moderna

Unidad 4 — Redes neuronales, visión por computador, secuencias, lenguaje y transformers.



Contenido

01 Fundamentos de redes neuronales

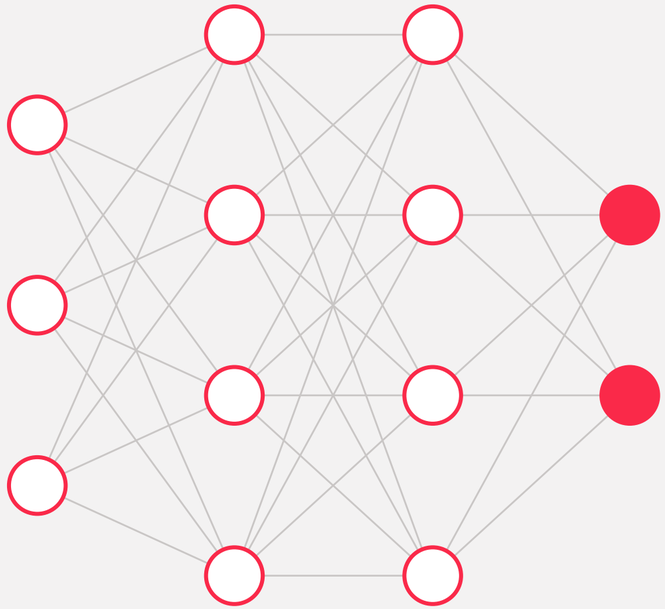
02 Visión por computador (clásica)

03 Redes neuronales convolucionales (CNN)

04 Modelos secuenciales (RNN, LSTM, GRU)

05 Procesamiento de lenguaje natural (PLN)

06 Transformers



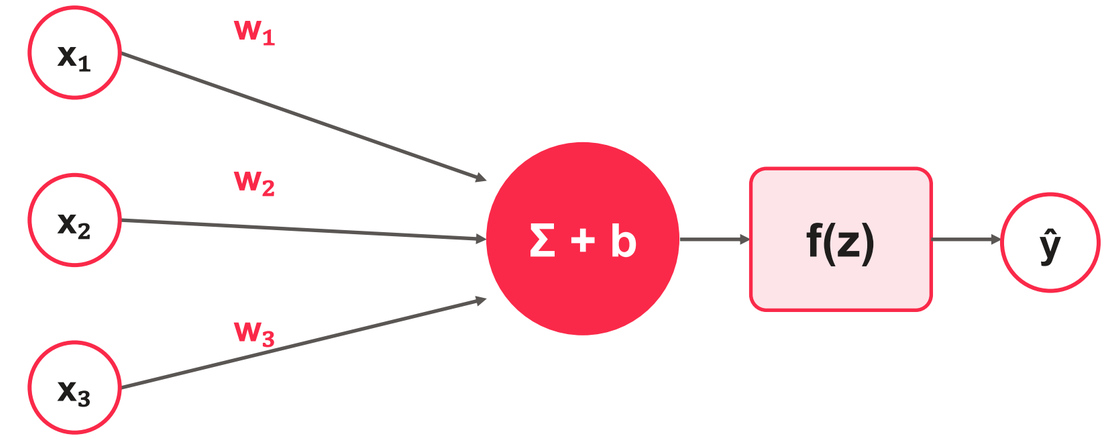
01

Fundamentos de Redes Neuronales

01

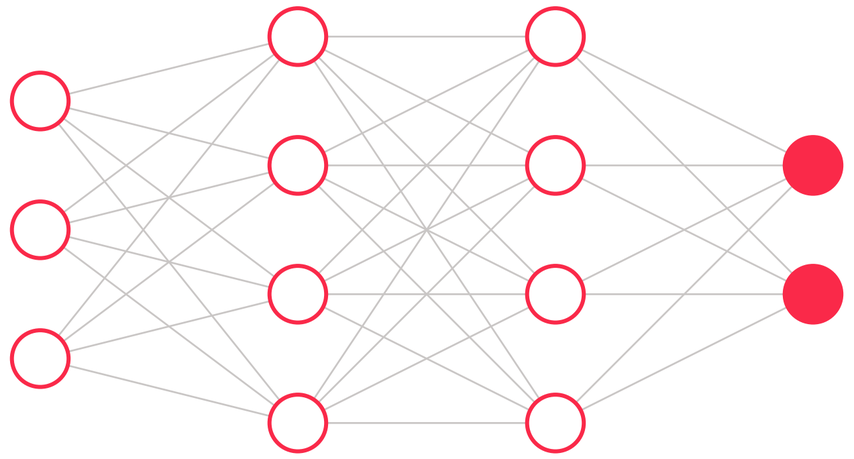
De la neurona artificial a la red neuronal

Una neurona artificial



$$z = w_1x_1 + w_2x_2 + w_3x_3 + b \qquad \hat{y} = f(z)$$

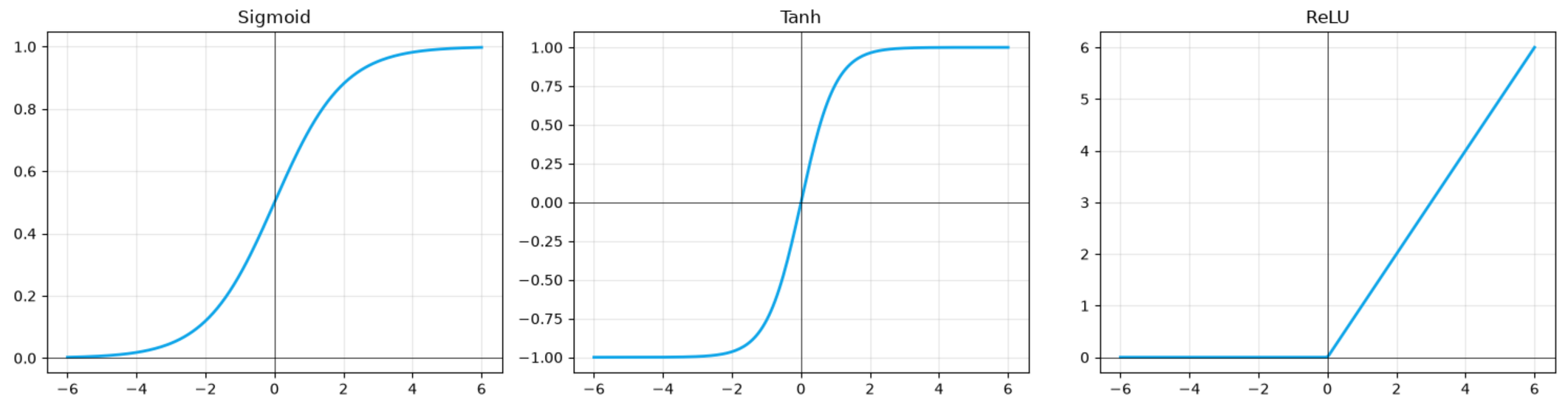
Una red = muchas neuronas en capas



Idea clave

Cada neurona suma sus entradas ponderadas, aplica una activación no lineal y pasa el resultado a la siguiente capa.

La neurona artificial y sus funciones de activación



1

Una neurona combina sus entradas linealmente (pesos + sesgo) y aplica una función de activación no lineal

2

Sin activación no lineal, apilar capas seguiría siendo equivalente a un único modelo lineal

3

Sigmoid y Tanh saturan en los extremos; ReLU es la más usada en capas ocultas por evitar ese problema para valores positivos

Entrenamiento: pérdida y descenso de gradiente

1 Función de pérdida

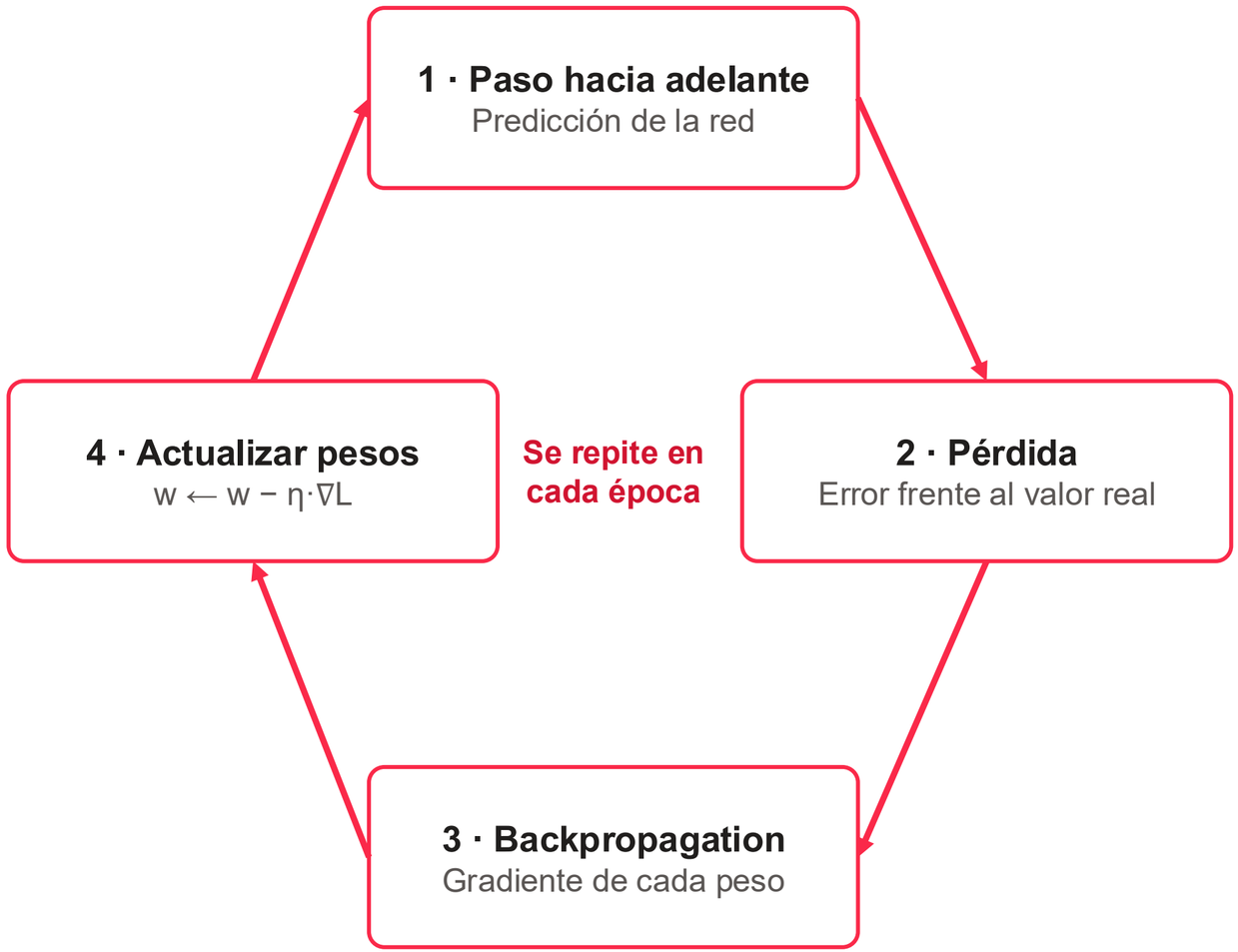
Mide qué tan lejos están las predicciones de los valores reales.

2 Descenso de gradiente

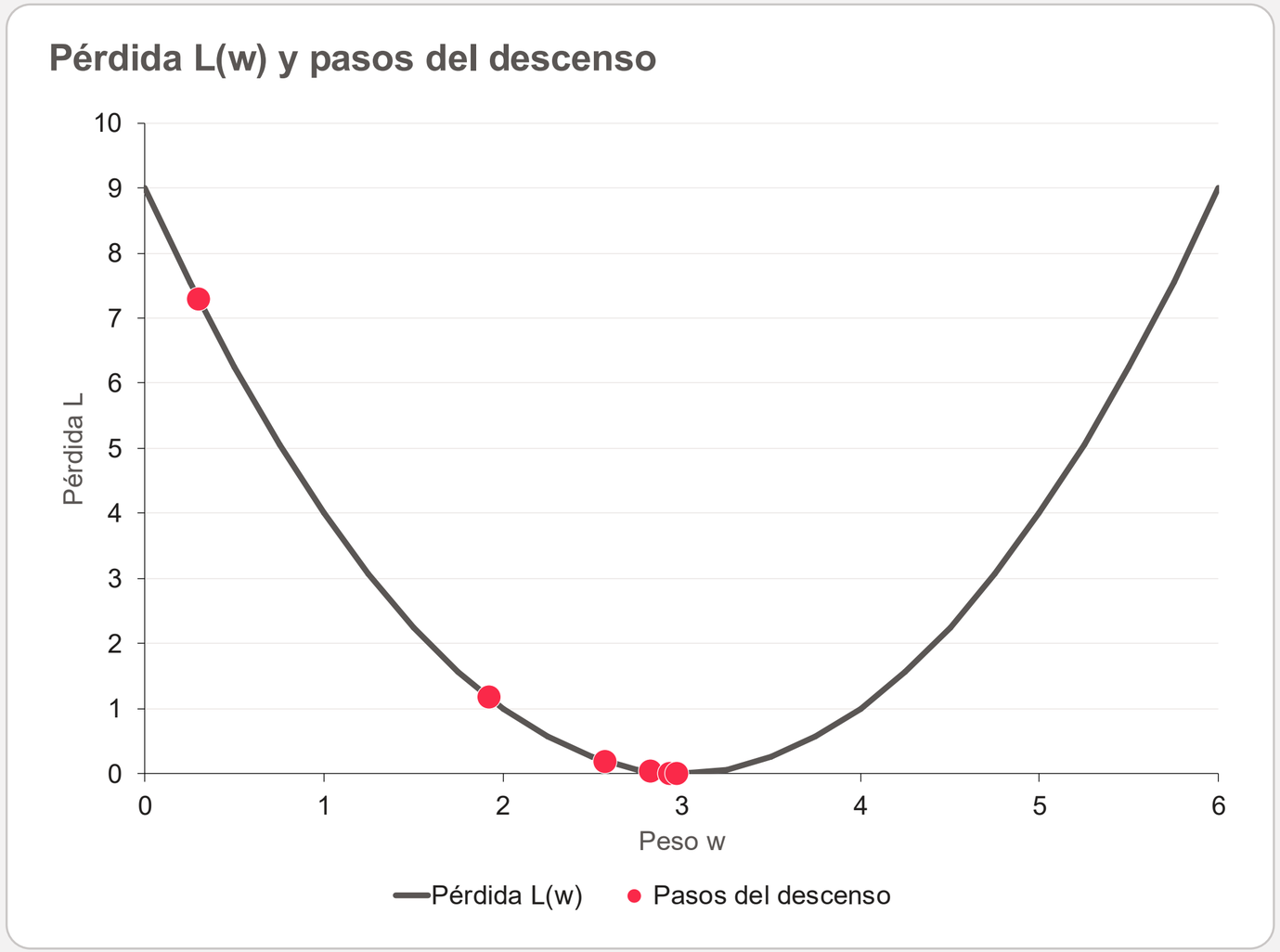
Ajusta los pesos en la dirección que más reduce la pérdida: el mismo algoritmo de la Unidad 3, ahora con miles de parámetros.

3 Backpropagation

Calcula el gradiente de la pérdida respecto a cada peso, capa por capa.



Descenso de gradiente: bajar hasta la mínima pérdida



1 Calcular el gradiente

La pendiente de la pérdida indica hacia dónde sube el error.

2 Dar un paso en contra

$w \leftarrow w - \eta \cdot \nabla L$. La tasa de aprendizaje η controla el tamaño del paso.

3 Repetir hasta converger

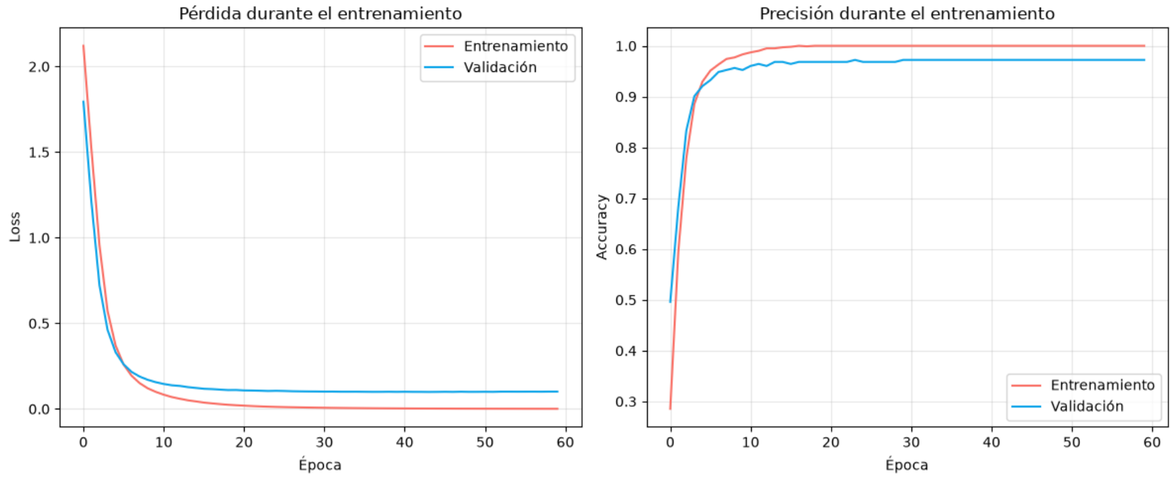
Cada paso baja la pérdida; cerca del mínimo la pendiente es casi cero.

Ejemplo real: clasificación de dígitos (Digits)

1797 imágenes de 8×8 píxeles

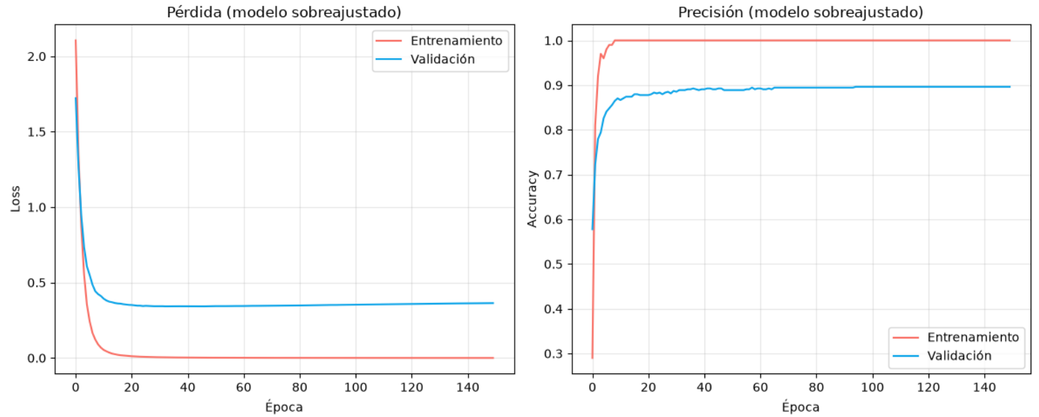
10 clases: dígitos del 0 al 9

97.8% de exactitud en el conjunto de prueba



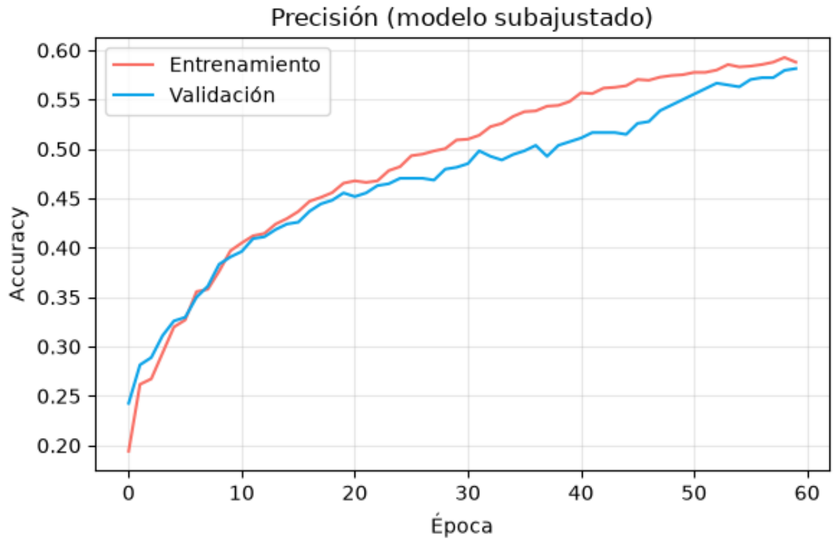
Sobreajuste y subajuste, con una red real

Sobreajustado (red grande, poca data)



Memoriza los datos. La pérdida de validación deja de bajar mientras la de entrenamiento sigue cayendo.

Subajustado (red demasiado pequeña)



No alcanza a aprender. Entrenamiento y validación quedan con una precisión baja.

02

Visión por Computador (Clásica)

02

Una imagen es una matriz de números

- Una imagen a color es un arreglo de alto × ancho × 3 canales (rojo, verde, azul)
- En escala de grises es un solo canal: cada píxel es un número entre 0 (negro) y 255 (blanco)
- OpenCV carga las imágenes en orden BGR, no RGB, una fuente común de errores si no se convierte antes de mostrarlas



Imagen → Matriz alto × ancho × 3 → **Valores de 0 a 255**

Un píxel son tres números: los canales R, G y B

Canal R (rojo)			
220	200	30	40
210	190	25	60
20	30	200	210
40	50	220	230

+

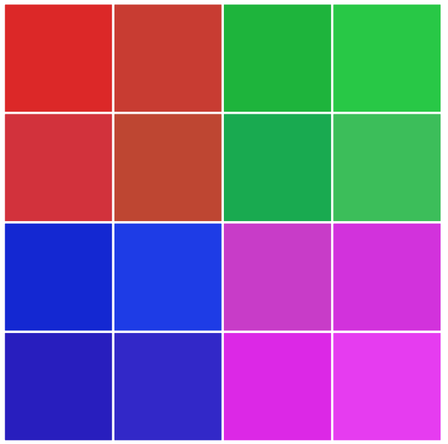
Canal G (verde)			
40	60	180	200
50	70	170	190
40	60	60	50
30	40	40	60

+

Canal B (azul)			
40	50	60	70
60	50	80	90
210	230	200	220
190	200	230	240

=

Imagen resultante



Cada número es la intensidad de ese color en un píxel: 0 = nada, 255 = máximo

Recuerda

En escala de grises basta 1 matriz (0 = negro, 255 = blanco); en color son 3 matrices apiladas: alto × ancho × 3.

Detección de bordes: Sobel



1

Sobel calcula el gradiente de intensidad en X e Y con un kernel 3×3, donde la intensidad cambia bruscamente, hay un borde

2

La magnitud del gradiente combina ambas direcciones en un solo mapa de bordes

Cómo se calcula un borde con Sobel

Ventana 3×3 en la imagen

10	10	10	200	200
10	10	10	200	200
10	10	10	200	200
10	10	10	200	200
10	10	10	200	200

La intensidad salta de 10 a 200

×

Kernel Gx

-1	0	1
-2	0	2
-1	0	1

Multiplicar y sumar

=

Respuesta

760

Gx en el centro: borde vertical fuerte

Kernel Gy

-1	-2	-1
0	0	0
1	2	1

Mismo proceso, otra dirección

Gx: bordes verticales → Gy: bordes horizontales → $G = \sqrt{(Gx^2 + Gy^2)}$ → **Mapa de bordes**

Binarización: umbral fijo vs. umbral de Otsu



1

Un umbral fijo (127) no se adapta a cada imagen; el método de Otsu calcula el umbral óptimo (97 en este caso) a partir del histograma de la imagen

2

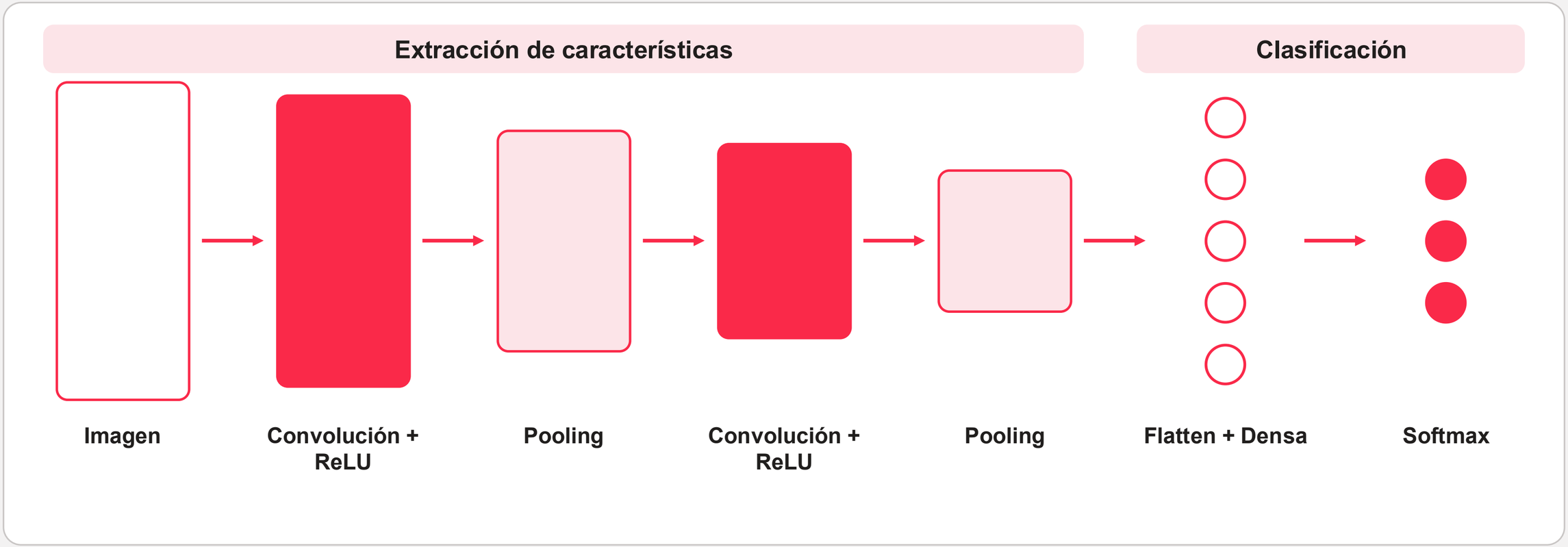
Otsu busca el punto que mejor separa dos grupos de píxeles (fondo y objeto)

03

Redes Neuronales Convolucionales (CNN)

03

Anatomía de una red convolucional



Idea clave

Las capas convolucionales detectan patrones, el pooling reduce el tamaño y las capas densas deciden la clase.

Convolución y pooling, paso a paso

Convolución: filtro 3×3 sobre la entrada

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Entrada 5×5

*

1	**0**	**1**
0	**1**	**0**
1	**0**	**1**

Filtro

=

4	**3**	**4**
2	**4**	**3**
2	**3**	**4**

Mapa de características

El filtro se desliza por la imagen: multiplica y suma en cada posición.

Max pooling 2×2

1	3	2	1
4	6	5	2
3	1	0	7
2	2	8	4

→

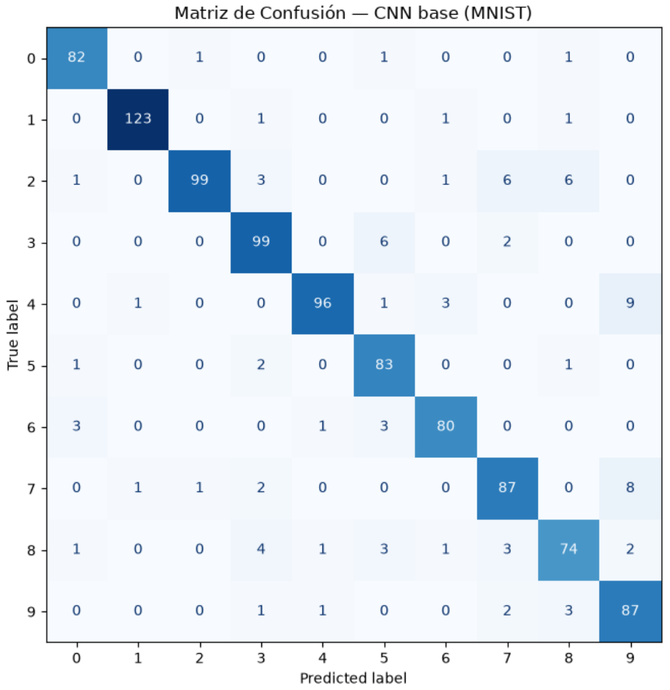
6	**5**
3	**8**

Se conserva el valor máximo de cada bloque: la imagen se reduce a la mitad.

Convolución: detecta patrones → Pooling: reduce el tamaño → **Menos cómputo, más robustez**

CNN base sobre MNIST: matriz de confusión real

- Confusiones reales más frecuentes: 4→9 y 2→7/8 — dígitos que también se parecen a simple vista.

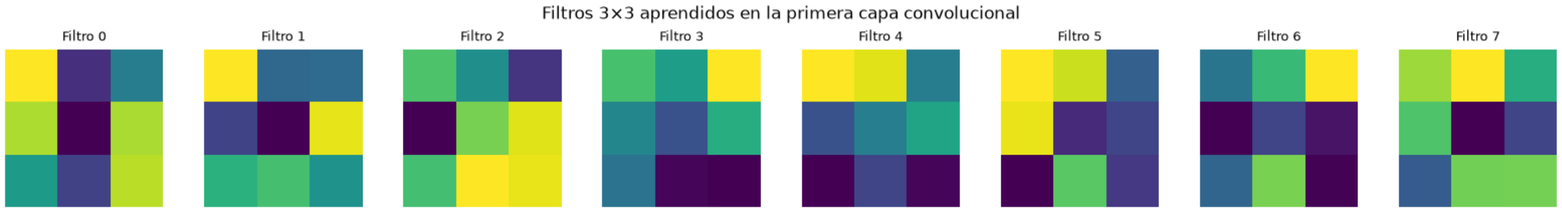


Idea clave

Lea la diagonal: son los aciertos. Fuera de ella están las confusiones.

Qué aprende cada filtro convolucional

Filtros 3×3 aprendidos (primera capa)



Mapas de activación sobre un dígito real

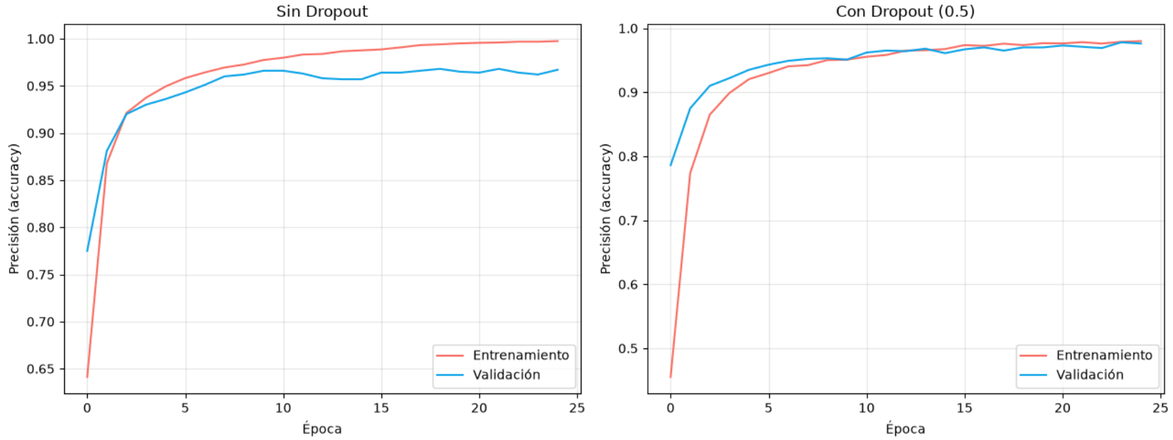


Idea clave

Cada filtro actúa como detector de patrones (bordes, esquinas, texturas); el mapa de activación muestra dónde aparece.

Dropout reduce la brecha entre entrenamiento y validación

- Sin dropout, la brecha entre entrenamiento y validación se mantiene; con dropout (0.5), ambas curvas terminan mucho más cerca.



Sin dropout

Dropout 0.5

Aumento de datos: más variedad, sin más imágenes



1

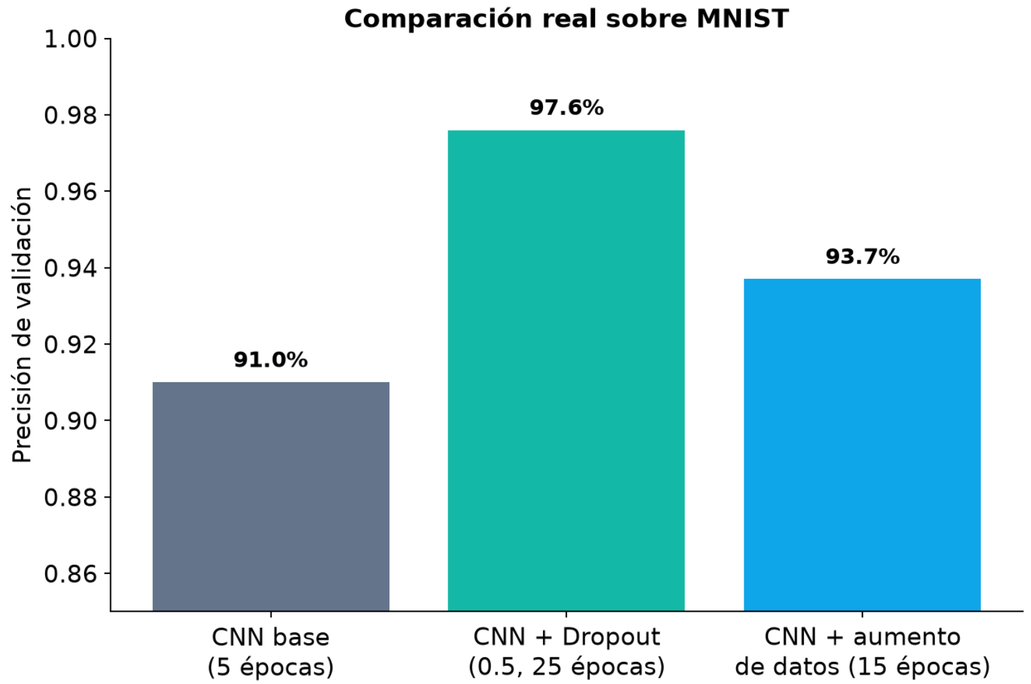
Pequeñas rotaciones, desplazamientos y zooms generan variantes realistas de la misma imagen de entrenamiento

2

El modelo ve efectivamente más ejemplos sin necesidad de recolectar más datos, reduciendo el sobreajuste

Comparación real, y transferencia de aprendizaje

- Dropout es, de las tres variantes, la que más mejora la validación en MNIST
- Transferencia de aprendizaje (MobileNetV2 congelado + clasificador nuevo) alcanza 88.9% en 3 clases de CIFAR-10
- Reutilizar una red preentrenada es mucho más rápido que entrenar una CNN desde cero



Idea clave

Dropout y transferencia de aprendizaje: dos formas de generalizar mejor.

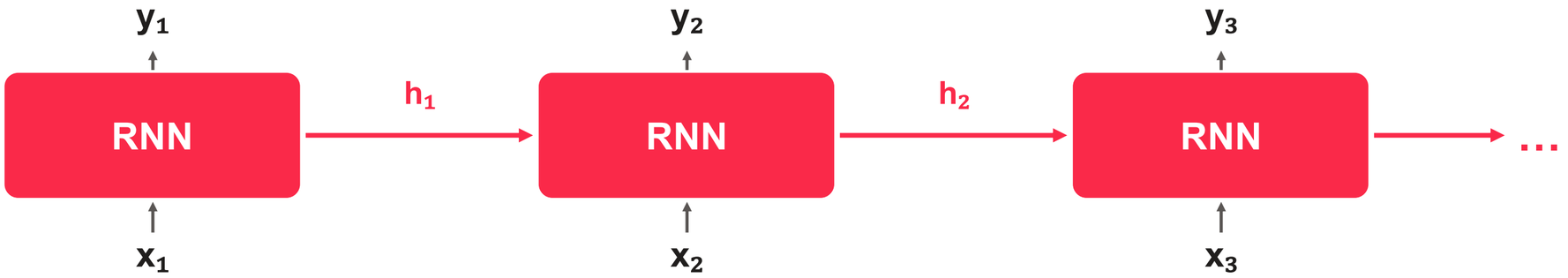
04

Modelos Secuenciales: RNN, LSTM y GRU

04

Memoria en secuencias: de la RNN a la LSTM

RNN: el estado oculto pasa de un paso al siguiente



LSTM: tres puertas controlan la memoria

Celda de memoria

Puerta de olvido
¿Qué se borra?

Puerta de entrada
¿Qué se guarda?

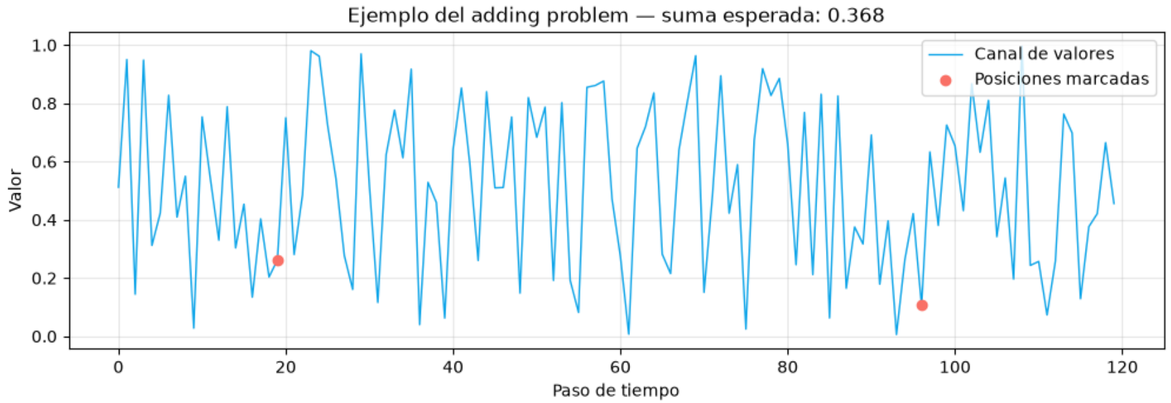
Puerta de salida
¿Qué se muestra?

GRU

Versión simplificada con 2 puertas (actualización y reinicio): rinde parecido con menos parámetros.

El "adding problem": dependencias largas

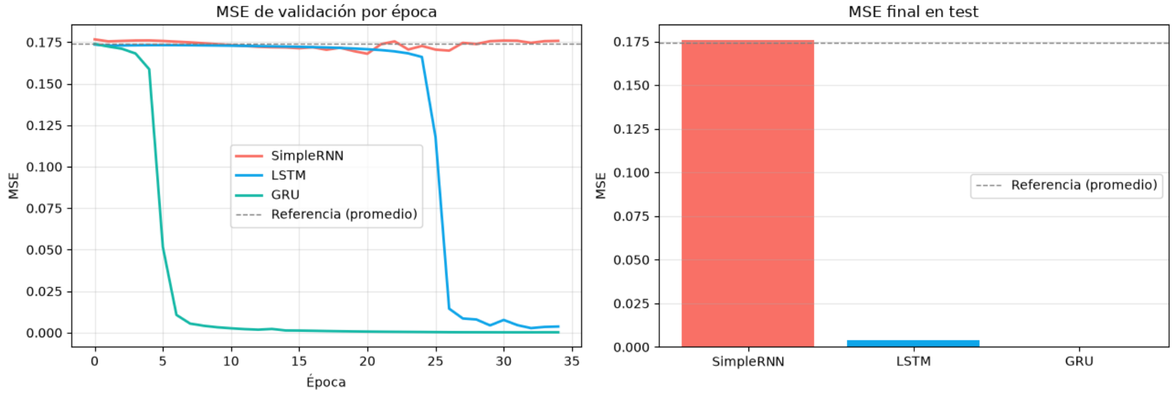
- Tarea sintética clásica: sumar los dos valores marcados, sin importar qué tan lejos estén entre sí en la secuencia
- Obliga al modelo a recordar información durante muchos pasos de tiempo, el punto débil de una RNN simple



Secuencia larga → Dos valores marcados → Recordarlos → **Sumarlos**

SimpleRNN no aprende; LSTM y GRU sí

- SimpleRNN se queda estancado en el error de referencia (predecir siempre el promedio) y no logra aprender la dependencia larga
- LSTM y GRU sí aprenden, gracias a sus "puertas" que controlan qué información recordar u olvidar



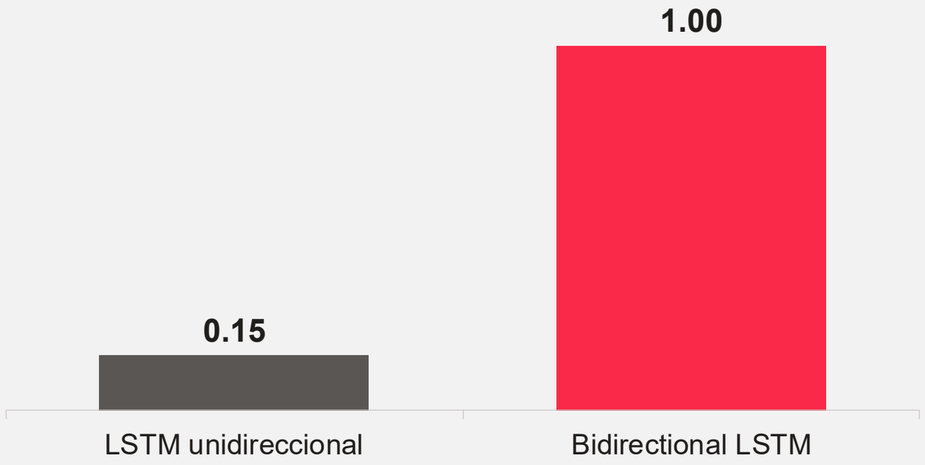
Idea clave

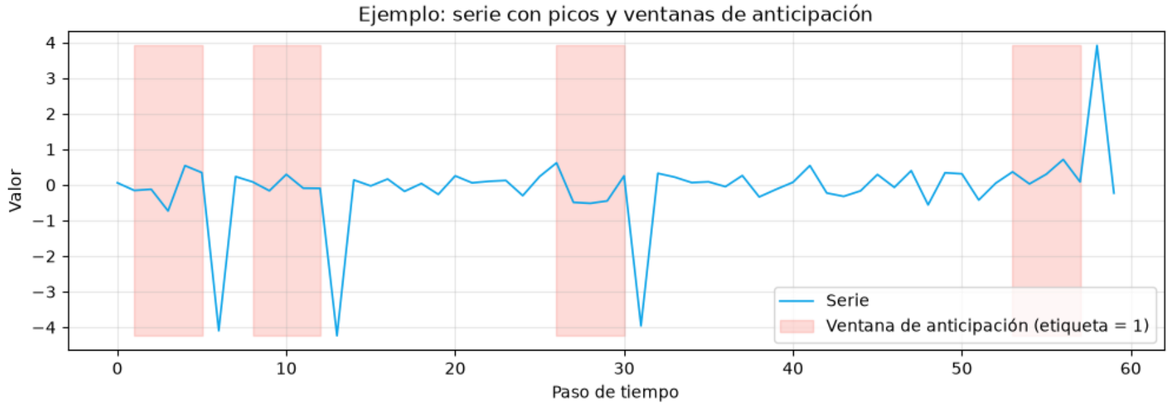
Las puertas de LSTM y GRU deciden qué recordar y qué olvidar.

Bidirectional LSTM: cuando el futuro también importa

- Tarea: anticipar un pico antes de que ocurra; requiere ver el contexto posterior
- Una LSTM unidireccional casi no lo logra; una Bidirectional LSTM lo detecta casi siempre

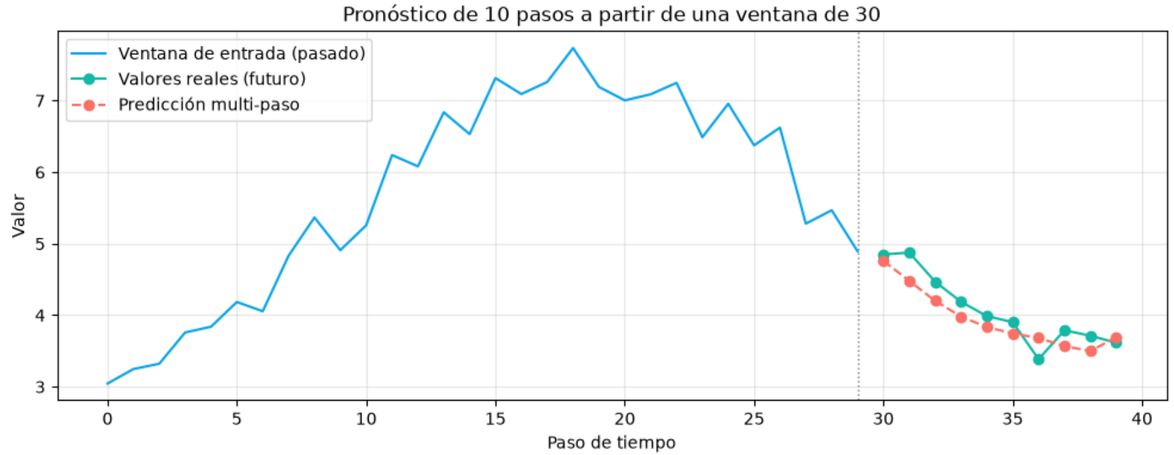
Recall en la detección de picos (≈)





Pronóstico multi-paso, sobre datos reales

- A partir de una ventana de 30 pasos pasados, el modelo predice 10 pasos futuros de una sola vez.



30 pasos pasados → Modelo → **10 pasos futuros**

05

Procesamiento de Lenguaje Natural (PLN)

05

De palabras a números

Del texto al modelo

Texto: «la película es genial» → Tokens: la · película · es · genial → **Vectores numéricos** → Modelo que clasifica

TF-IDF

Cada documento es un vector de frecuencias de palabras, ponderadas por qué tan distintiva es cada una.

0	0	**0.8**	0	**0.3**	0	**0.5**	0

Embeddings

Cada palabra es un vector denso aprendido: palabras de significado similar quedan cerca entre sí.



Datos

Dataset de trabajo: reseñas de películas de IMDB, etiquetadas como positivas o negativas.

TF-IDF + Regresión Logística: matriz de confusión real

TN · Negativa bien clasificada

400

FP · Falsa positiva

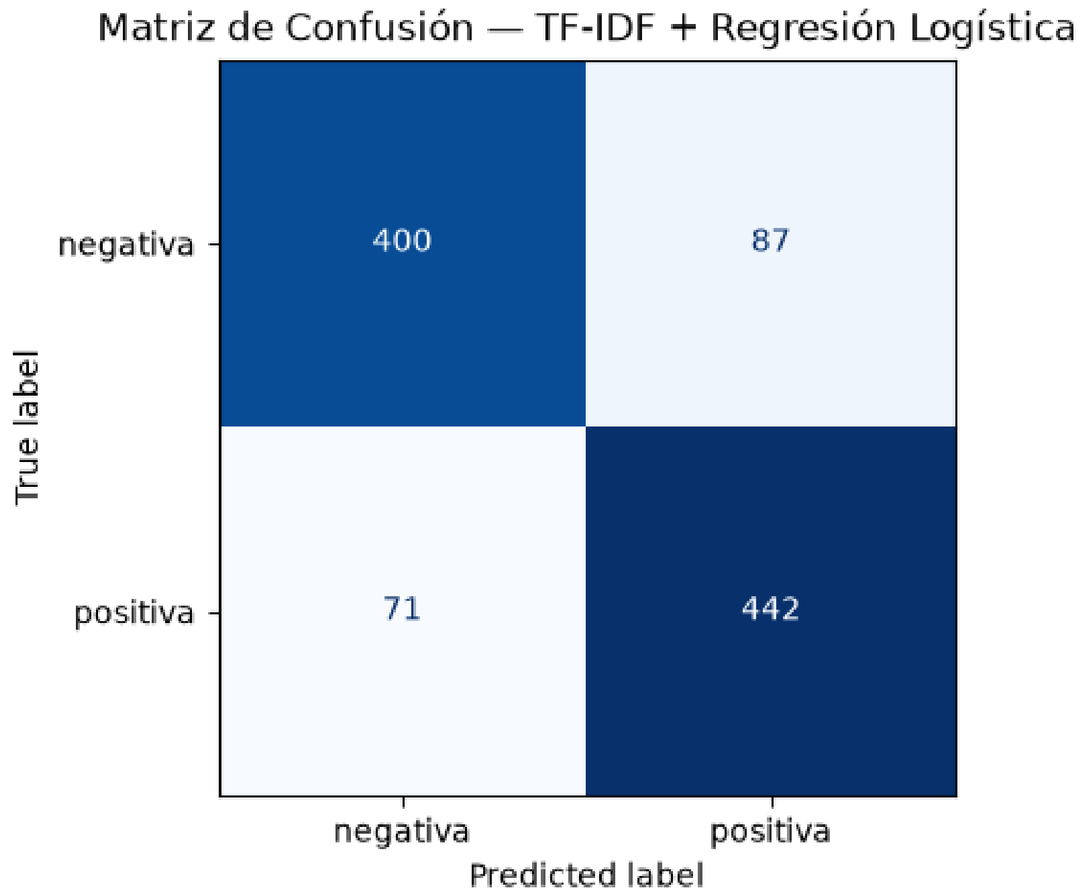
87

FN · Falsa negativa

71

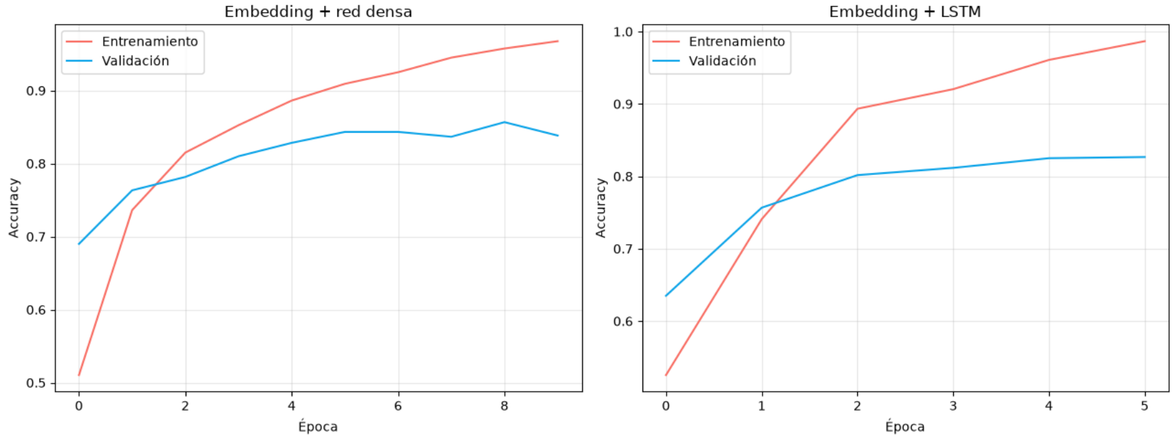
TP · Positiva bien clasificada

442



Embedding + red densa vs. Embedding + LSTM

- Ambos modelos muestran una brecha clara entre entrenamiento y validación: sobreajuste real, con relativamente pocos datos de entrenamiento
- La LSTM no supera aquí al enfoque más simple. Más complejidad no siempre gana



Idea clave

Más complejidad no siempre gana: compárala con un modelo simple.

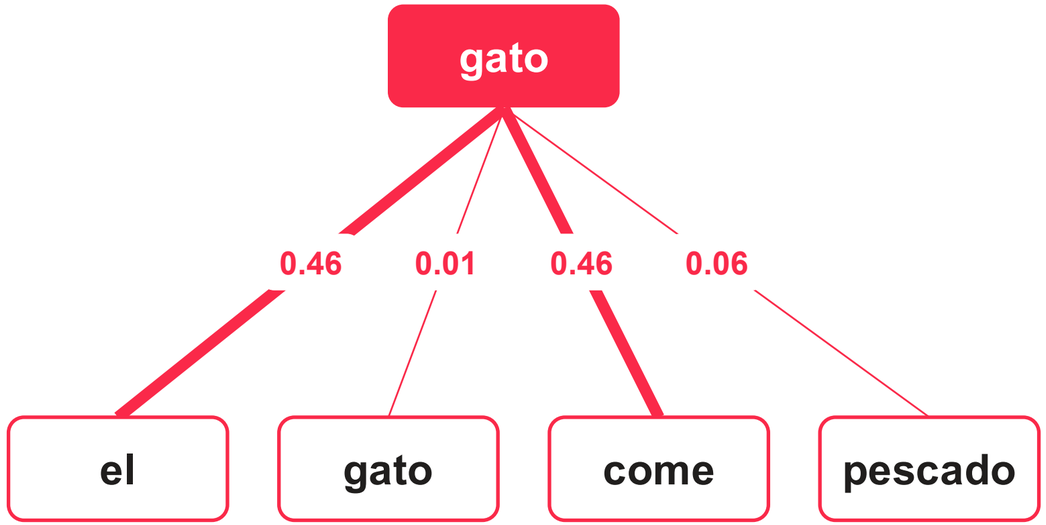
06

Transformers

06

Self-attention paso a paso

La palabra «gato» reparte su atención



Más grueso = más atención

1 Q, K, V

Cada palabra genera una consulta (Q), una clave (K) y un valor (V).

2 Puntajes y softmax

$Q \cdot K^T$ mide la afinidad entre palabras; softmax la convierte en pesos que suman 1.

3 Mezclar valores

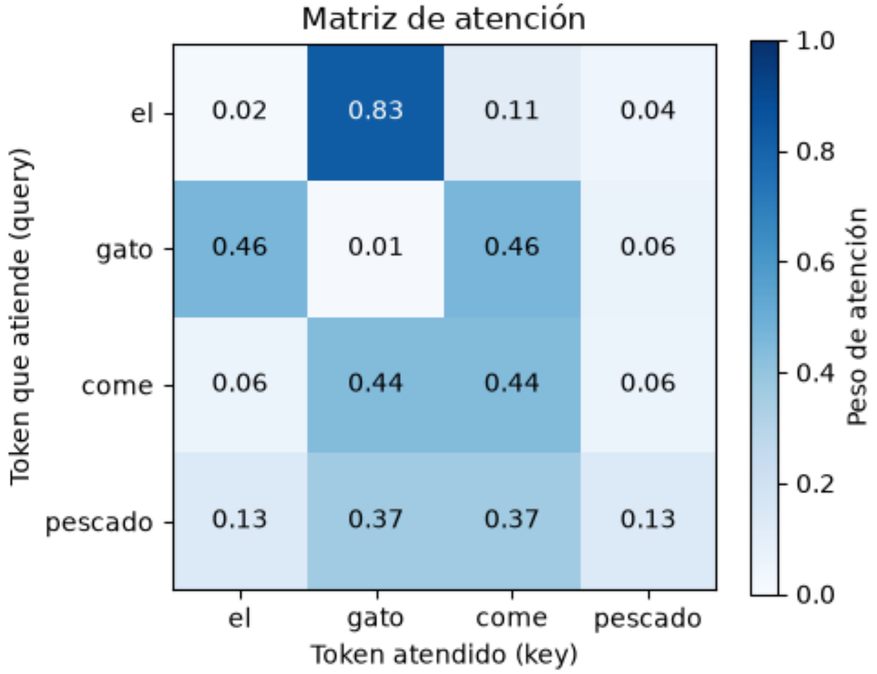
La salida es la suma de V ponderada por esos pesos.

Fórmula

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

Self-attention: cada palabra "mira" a las demás

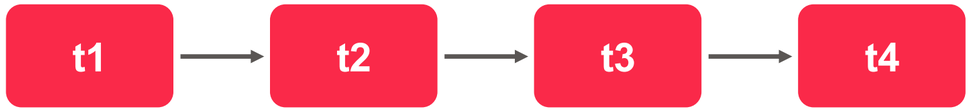
- Calculado a mano con NumPy: "gato" atiende fuertemente a "el", el sujeto de la oración.



Idea clave **Cada fila de pesos suma 1: es una distribución de atención.**

De RNN a Transformers

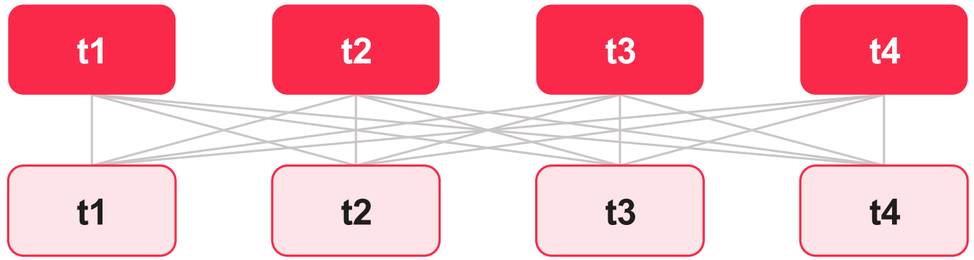
RNN / LSTM: paso a paso



Cada paso espera al anterior

- No se pueden paralelizar
- Les cuesta recordar dependencias muy largas

Transformer: todo a la vez



Cada palabra atiende a todas las demás

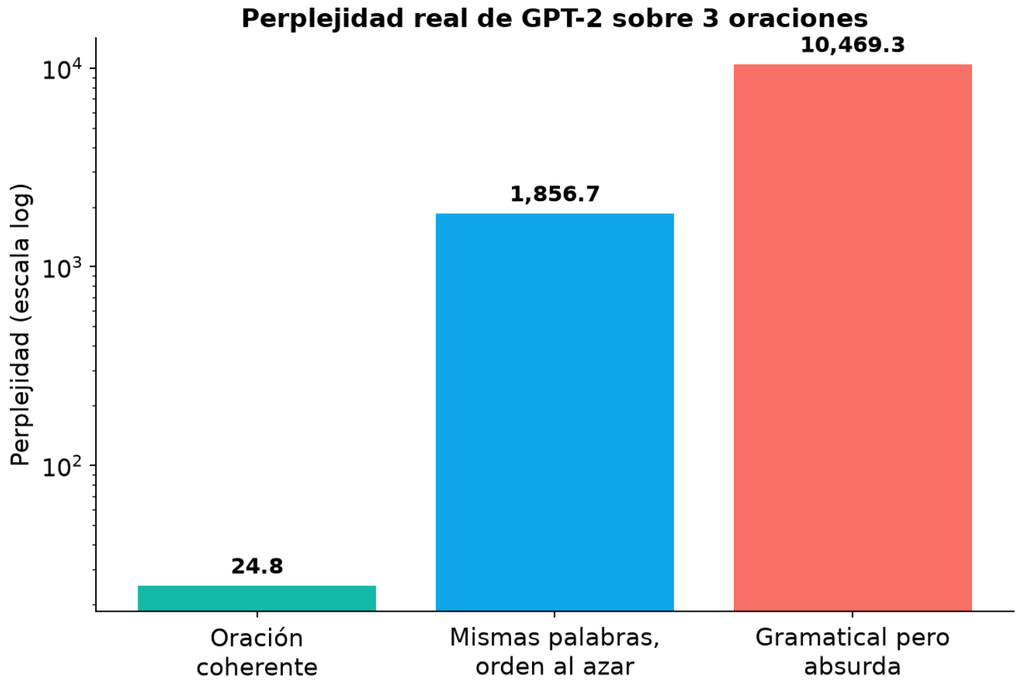
- Self-attention calcula en paralelo cuánto atiende cada palabra a las demás
- Es la pieza central de modelos como BERT ó GPT

Idea clave

Reemplazar la recurrencia por atención elimina el cuello de botella secuencial.

Evaluar un modelo generativo: perplejidad

- Perplejidad = qué tan "sorprendido" está el modelo al leer un texto (menor es mejor)
- GPT-2 (real): 24.8 en una oración coherente, pero miles de veces mayor en una versión con las mismas palabras desordenadas
- Útil para modelos generativos, donde no existe una única "respuesta correcta" que comparar como en clasificación



Idea clave

Perplejidad menor significa que el modelo está menos sorprendido.

RECURSOS

Resumen de la Unidad 4

	Tema	Notebook del curso
01	**Fundamentos de redes neuronales**	1-fundamentos-redes-neuronales.ipynb
02	**Visión por computador clásica**	2-vision-por-computador.ipynb
03	**Redes convolucionales (CNN)**	3-redes-convolucionales.ipynb
04	**Modelos secuenciales (RNN/LSTM/GRU)**	4-modelos-secuenciales.ipynb
05	**Procesamiento de lenguaje natural**	5-procesamiento-lenguaje-natural.ipynb
06	**Transformers**	6-transformers.ipynb

¡Gracias!

De la neurona artificial a los transformers — la base de todo lo que viene en la Unidad 5 (agentes de IA).

